目录

蓝牙打印机开发资料.....2
1. 蓝牙打印机.....2
1.1 蓝牙打印机说明.....2
1.2 项目需求介绍.....3
1.3 硬件介绍.....4
1.4 软件环境介绍.....5
2. ESP32 基础开发.....13
2.1 IO 口操作.....14
2.2 串口操作.....17
2.3 蓝牙操作.....19
3. 蓝牙打印机功能开发.....22
3.1 打印头控制.....22
3.2 缺纸检测.....32
3.3 电压检测.....33
3.4 上位机通信.....34
3.5 喵喵机 APP 使用.....43

蓝牙打印机开发资料

1. 蓝牙打印机

1.1 蓝牙打印机说明

蓝牙热敏打印机是一种便携式打印设备，利用热敏技术来打印文字、图像或条形码。它们通常与智能手机、平板电脑、笔记本电脑等设备连接，通过蓝牙无线技术进行通信和数据传输。这种打印机广泛应用于零售、仓储、快递、餐饮、票务等行业，以及需要在移动场景下进行打印的工作。



以下是蓝牙热敏打印机的一些主要特点和优势:

便携性：蓝牙热敏打印机体积小巧，重量轻，易于携带。这使得它们成为在移动场景下打印标签、收据或票据的理想选择，如快递员递送货物时的签收单、餐厅打印账单等。

无需墨水或色带：蓝牙热敏打印机使用热敏技术，无需墨水、色带或碳带。它们通过加热打印头来打印内容，因此无需耗材更换，维护成本较低。

简单易用：连接蓝牙热敏打印机通常只需进行简单的配对步骤，然后就可以开始打印。

高速打印：蓝牙热敏打印机通常能够快速完成打印任务，适用于需要迅速获取打印输出的场景。

多平台兼容：这些打印机通常支持多个操作系统，如 Android、iOS、Windows 等，因此可以与各种不同类型的设备连接和使用。

低噪音：相比传统的针式打印机，热敏打印机工作时产生的噪音较小，不会干扰周围环境。

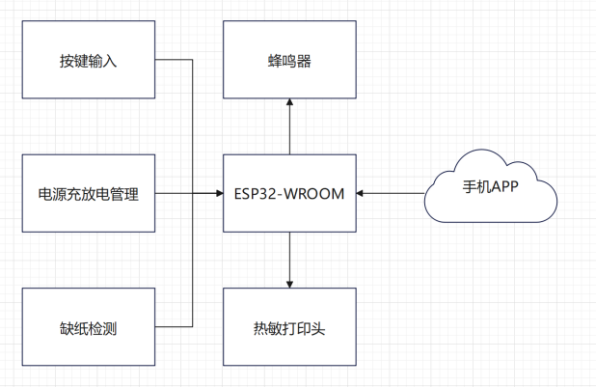
节能环保：由于无需使用墨水或色带，蓝牙热敏打印机在使用时不会产生过多的废弃物，对环境更加友好。

1.2 项目需求介绍

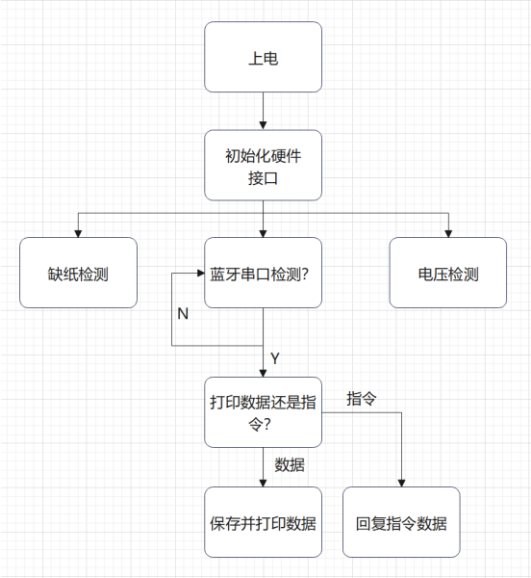
1.2.1 蓝牙打印机功能分析

- 按键走纸功能：通过按钮控制打印头走纸，便于安装热敏纸
- 缺纸检测功能：能够检测是否装入热敏纸，避免缺纸打印烧毁打印头
- LED 功能：能够提示开始打印和缺纸报警
- 蓝牙功能：和上位机进行蓝牙通信
- 热敏打印功能：通过热敏打印头，将蓝牙收到的数据转印到热敏纸上
- 电源检测功能：实时检测电池电压

1.2.2 蓝牙打印机实验平台硬件设计框图



1.2.3 蓝牙打印机实验平台软件设计流程图



1.3 硬件介绍

蓝牙打印机的主控采用 ESP32 作为核心，ESP32 是一款广受欢迎的低成本、高性能的 Wi-Fi 和蓝牙双模单片机。它支持广泛的应用领域，如物联网（IoT）、嵌入式系统和无线通信等。打印头采用 JX-700-48 热敏打印头，实现走纸、打印、以及缺纸检测和打印头温度检测的功能。

1， 主控芯片：

型号： ESP32-WROOM；核心：Xtensa 双核 32 位 LX6 微处理器；主频 240MHZ;FLASH:448KB;



ESP32-WROOM-32E



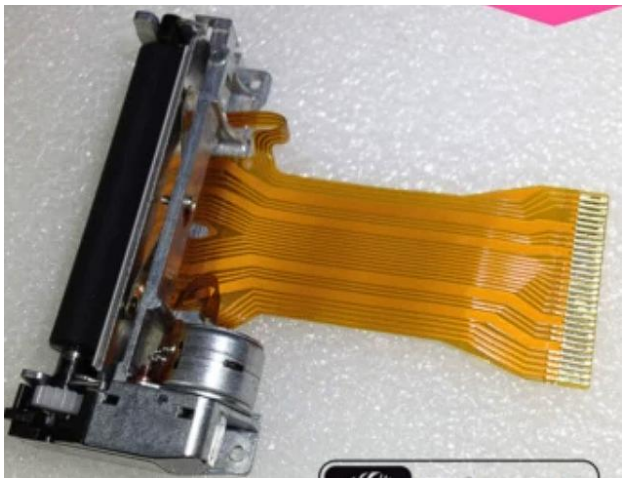
ESP32-WROOM-32UE

2， 打印头：

型号：JX-700-48；打印点数：384 点/行；打印宽度：48mm；集成光电式缺纸检测和温度检测；

3， 步进电机（打印头自带）：

类型：两相四线；步距角：9°；不近距离：0.0625mm



1.4 软件环境介绍

对于 ESP32 的开发常用的开发环境有以下几种可选;

Arduino IDE: Arduino IDE 是一种简单易用的开发环境, 适用于初学者和快速原型开发。为了在 Arduino IDE 中使用 ESP32，您需要添加 ESP32 的开发板支持。可以通过添加 ESP32 的 Arduino 核心来编写和上传代码。

PlatformIO: PlatformIO 是一个跨平台的开发生态系统, 支持多种硬件平台, 包括 ESP32。它是一个插件化的开发环境, 支持多个集成开发环境(IDE), 如 Visual Studio Code 和 Atom。PlatformIO 提供了丰富的库和示例，使得开发过程更加高效。

Espressif IDF：Espressif IoT Development Framework（IDF）是 Espressif 官方提供的开发框架。IDF 提供了更加底层和全面的控制，适合专业开发人员和需要更多自定义功能的项目。IDF 是用 C 语言编写的，提供了丰富的 API 和示例。

在此次项目中我们采用 VSCode+PlatformIO 的形式进行开发, 一方面可以使用 Arduino 提供的库文件极大地提高开发的效率, 另一方面弥补了 ArduinoIDE 不方便查看源码的短板。

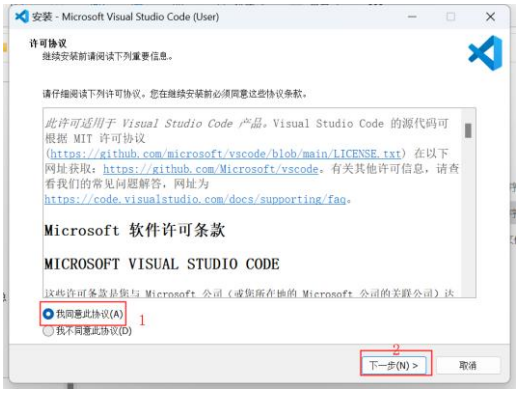
1.4.1 VS-Code 安装

步骤 1：下载 VSCode 安装程序

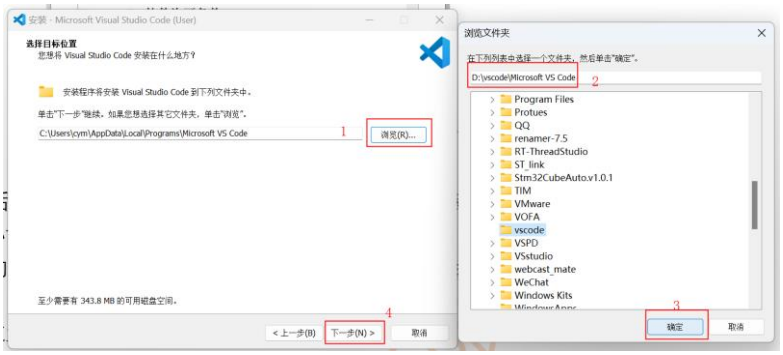
找到项目资料中提供的 VSCode 软件安装包，双击打开：

VSCodeUserSetup-x64-1.80.0

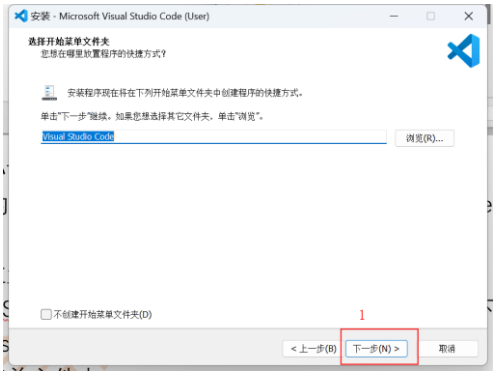
步骤 2：运行安装程序，弹出的第一个界面中，勾选同意该协议，然后点击下一步；



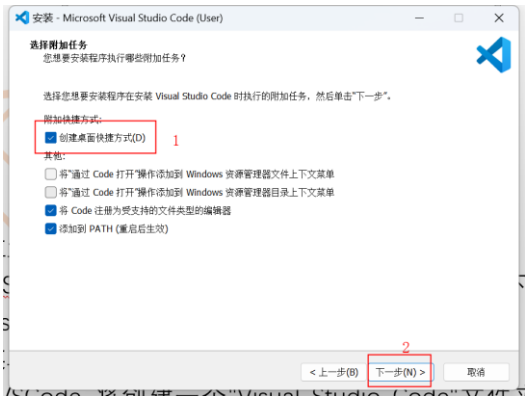
步骤 3：开始安装，在第二个界面中，选择自己的安装路径，不要在 C 盘，避免安装路径出现中文，然后点击下一步；



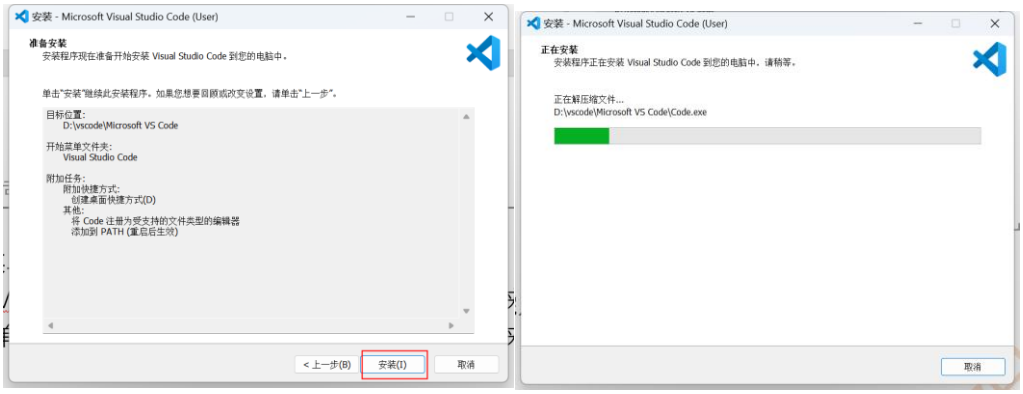
步骤 4：第三个界面中，直接点击下一步即可；



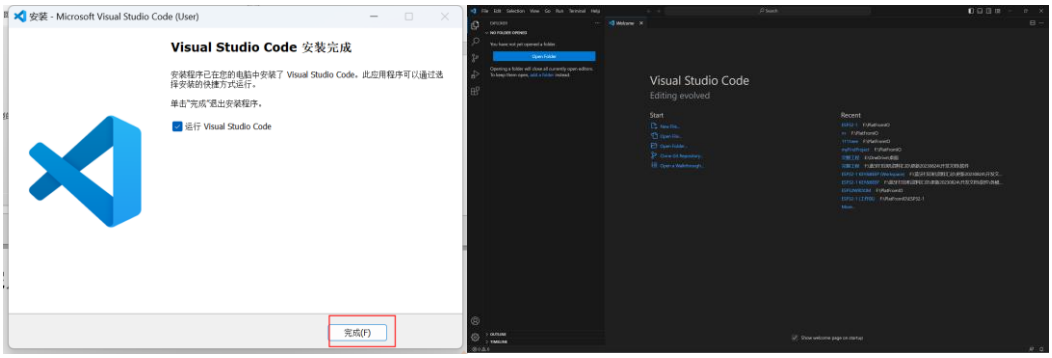
步骤 5：勾选创建桌面快捷方式，然后点击下一步；



步骤 6：直接点击下一步即可开始安装

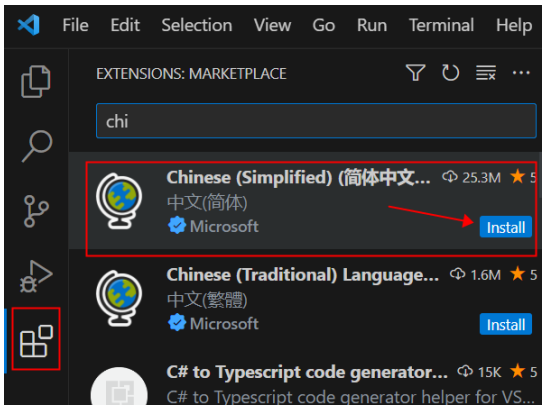


步骤 7：等待安装完成，点击完成即可自动打开该软件，出现如下界面



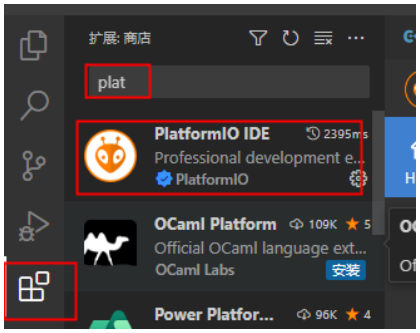
恭喜！您已经成功安装了 Visual Studio Code。现在，您可以打开 VSCode 开始编写代码和使用各种插件扩展其功能。

步骤 8：相关插件安装，点击插件按钮，在搜索框输入自己想要的插件，点击 install 即可安装

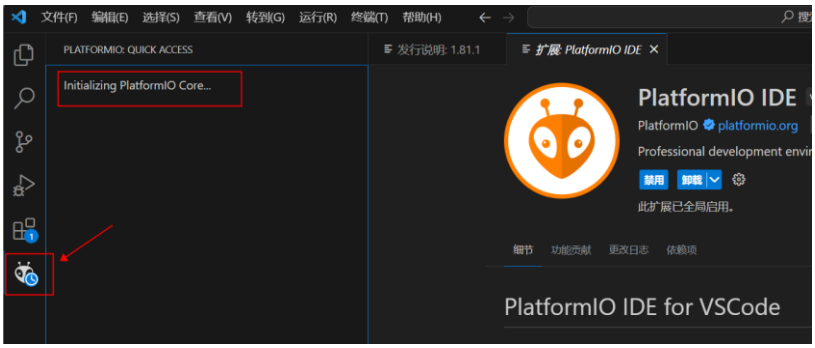


1.4.2 platform 插件安装

在拓展栏里搜索 PlatFromIO，找到一个蚂蚁头像的插件，点击安装即可；



插件添加很快，会在右侧出现 PlatFromIO 插件图标，点击进去之后会发现，还需要再进一步下载相关的插件，等待下载完成即可；



1.4.3 创建第一个 ESP32 工程

我们先对 PlatformIO 界面做一下大体介绍:

下图中所以红框 1 内所包含的是对工程的相关操作，如编译、上传、清除中间文件等，其功能和最下方红框 5 中所包含的功能图标一致

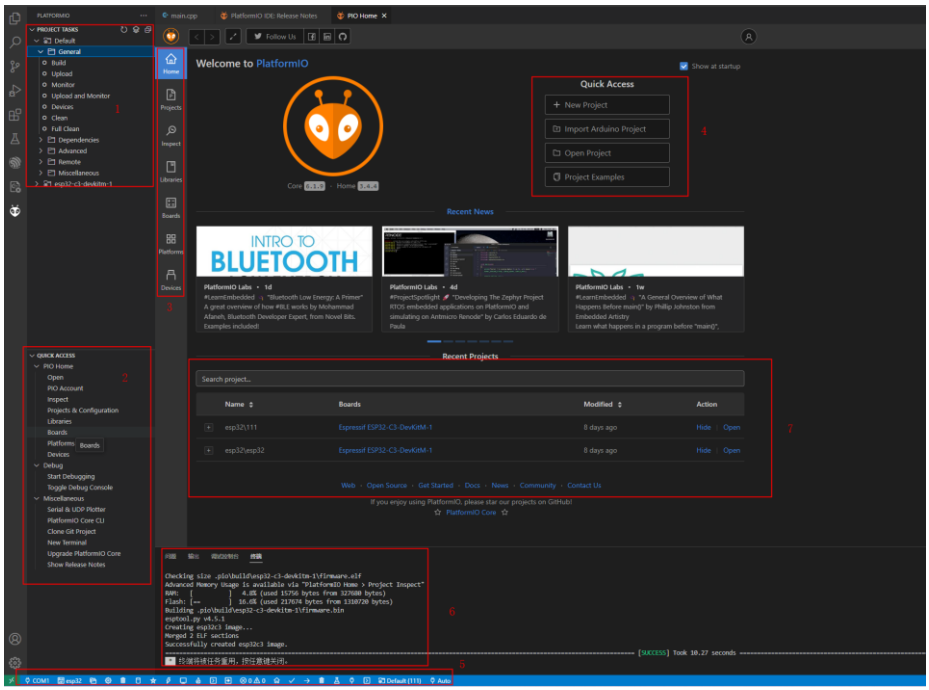
下图中所示方框 2 内所包含的是 PlatformIO 相关的功能，如打开工程 PlatformIO 主界

面、工程管理、库文件搜索、开发板搜索等，其功能和主界面中的红色框 3 内的功能一致。

下图所示红框 4，是创建工程和导入工程以及打工程的快捷按钮。

下图所示红框 6，是开发过程中编译信息串口中端信息的输出窗口，如果编译产生错误可以在该处查找原因。

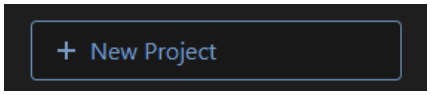
下图所示红框 7，是历史创建的工程显示的位置，如果之前创建的有工程就会在该位置显示。



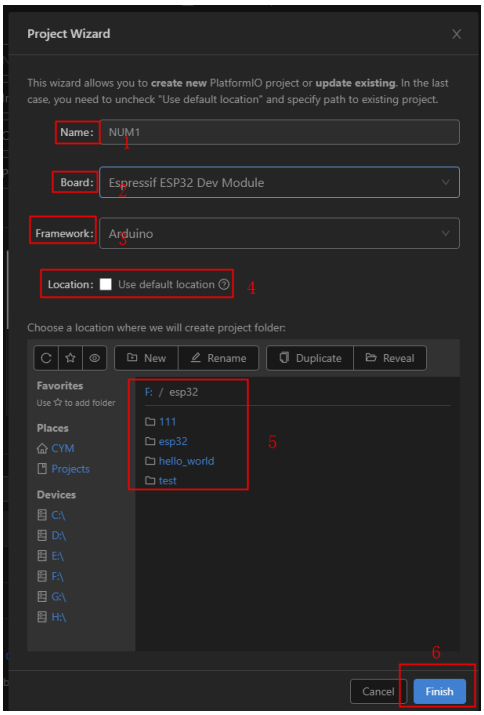
关于 PlatfromIO 的具体细节介绍可以参考其官网中提供的安装文档，链接如下：
[PlatformIO IDE for VSCode — PlatformIO 最新文档]

熟悉基础界面和操作之后，接下来我们开始创建我们的第一个 Arduino 框架下的 ESP32 工程：

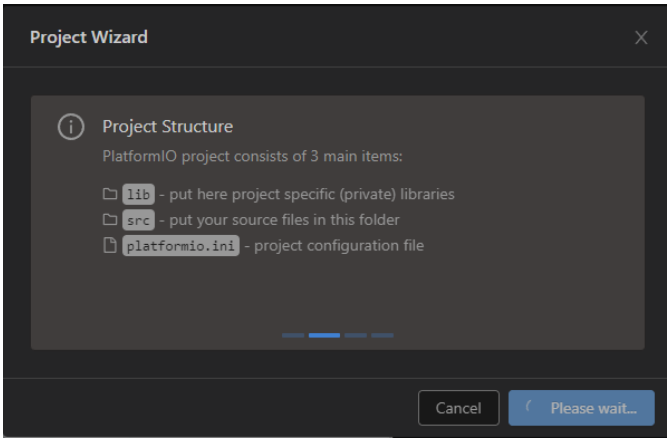
第一步：点击创建工程按钮



第二步：里我们使用的是 ESP32-WROOM 所以选择 ESP32 Dev Module 即可，3 为开发的框架，我们采用 Arduino 框架，4 为工程路径，默认是勾选的，采用的是默认路径在 C 盘，为了方便后需使用，我们一般取消勾选，然后在红框 5 处选择一个自己合适的纯英文路径即可。配置完成点击 Finish 即可。



第三步：第一次创建工程会下载 ESP32 开发环境的相关文件，而且由于网络环境问题，下载速度较慢，一般需要 1 小时左右，需耐心等待。我们也可以通过离线的方式将下载好的软件包替换到自己的插件安装路径即可。



离线安装方式：

首先关闭 VScode，提供的资料里有一个 PlatformIO 插件文件夹，这里是已经下载过相关文件的，组要替换掉自己工程里的文件。

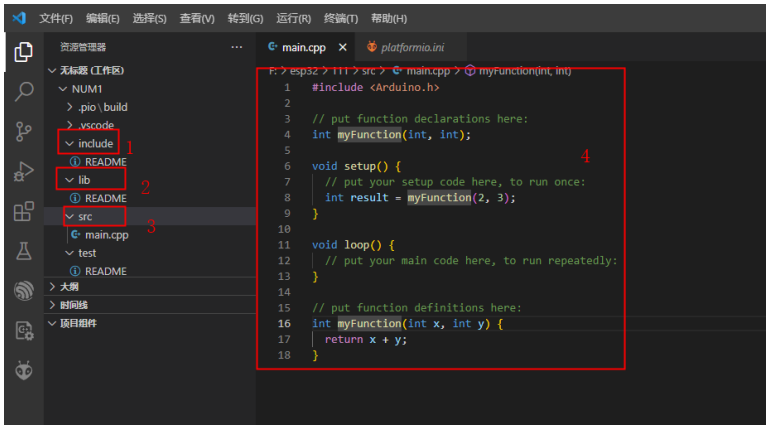


找到自己电脑 C 盘的用户文件夹，里边一般会有三个文件夹，找到以自己用户名命名的文件夹，然后在改文件夹里找到.platfromio 文件夹。用资料里提供的文件替换自己 C 盘里的文件夹。



替换完成之后，重新打开 VScode，软件会自动重新安装 PlatFromIO 插件，等待安装完成之后，重复上几步的操作

第四步：创建完成之后，会直接进入代码界面：默认生成的文件夹里有 include 用来存放我们的头文件，lib 文件夹用来存放第三方库，src 用来存放我们源文件。至此我们的工程就已经创建完成了，接下来就可以去写代码了。



1.4.4 软件基础使用

1，Arduino 框架下的编程方式

在使用 Arduino 框架进行开发时，首先我们需要先包含一个 Arduino.h 的头文件，这就和做 STM32 开发时需要先包含 stm32f10x.h 是一样的，主要是对一些后续用到的库的声明。

```
#include <Arduino.h>
```

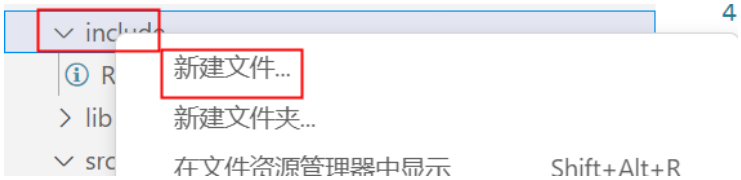
Arduino 代码框架有两部分组成，一个是 Setup 函数，一个是 loop 函数。Setup 函数中一般用于放置初始化相关的代码，比如 IO 口模式配置，串口配置等等……，loop 函数会一直循环执行，类似于 STM32 编程时 main 函数中的 while(1)，所以我们可以在其中放置需要循环执行的代码。上电之后程序执行的流程会先执行 setup 函数，然后在循环执行 loop 函数。

我们也可以在 setup 函数末尾加上 while(1)；也可实现同样的效果。

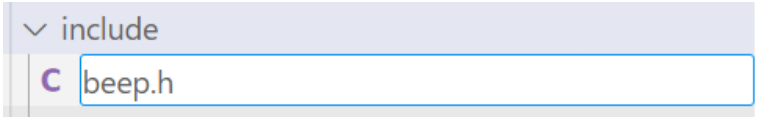
```
ESP32-1 > src > main.cpp > ...
1   #include <Arduino.h>
2   /*基础测试工程*/
3
4   void setup() {
5   //初始化代码
6   }
7
8   void loop() {
9   //循环执行的代码
10  }
```

2，新建文件和删除文件

随着功能越来越多，我们在这里仍然可以采用模块化的编程方式来管理工程。将鼠标移动到 include 目录上右键选择新建文件，就可以创建一个头文件；

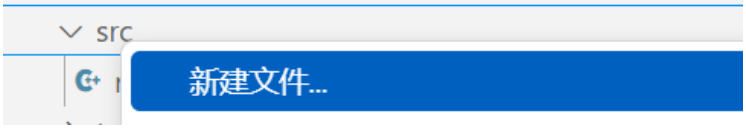


在文本框中输入头文件的名字然后按回车键即可：

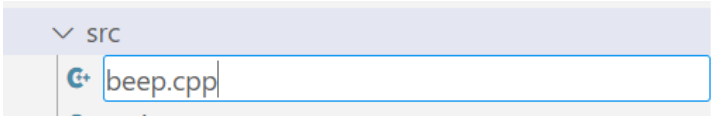


这样我们就可以创建一个模块的头文件出来，而且 include 这个目录已经默认包含在了编译的头文件路径中，我们不需要额外去修改。

将鼠标移动到 src 目录上，右键选择新建文件，就可以创建一个源文件。



需要注意的是 Arduino 编程时，库文件大多采用 C++的编码风格，所以我们在创建对应模块的源文件时也需要以 C++的格式命名，以.cpp 结尾，然后回车。



对应的源文件和头文件里的内容和格式跟 STM32 编程时基本一致。

3，PlatfromIO 配置文件修改

工程创建完成之后会自动生成一个配置文件 paltfromio.ini，为方便串口调试，我们需要在其中添加上串口波特率的配置，在原有代码基础上添加上：monitor_speed = 115200 ，即可设置串口波特率为 115200。

platformio.ini

```
11  [env:esp32dev]
12  platform = espressif32
13  board = esp32dev
14  framework = arduino
15  monitor_speed = 115200
16
```

4，编译和下载

编译和下载可以点击 VSCode 左下角的快捷键实现：



1， PlatformIO 主界面

2， 编译按钮

3， 程序下载按钮

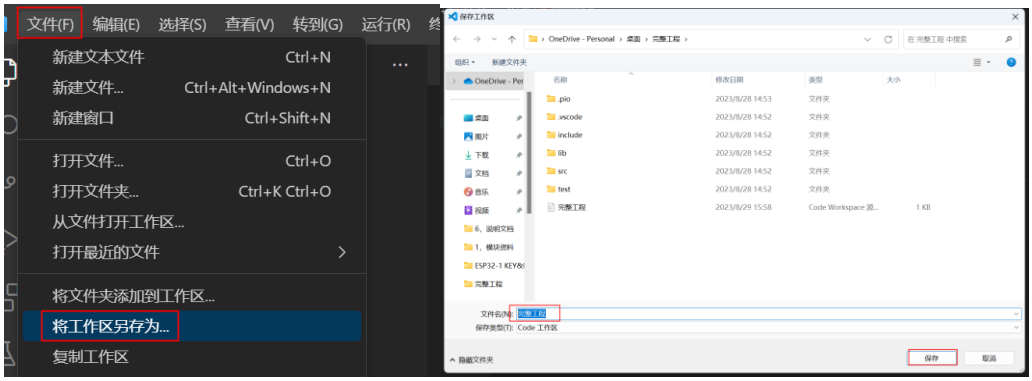
4， 清楚编译产生的中间文件

5， 打开串口监视器，类似于串口助手

6， 打开新的终端

5，工程打开和关闭

保存自己的工程，点击文件，选择将工作区另存为，输入自己的工程名字即可。打开工程时，只需要双击自己保存的工作区，就会自动打开。



2. ESP32 基础开发

本章我们将介绍在 ESP32 环境下使用 Arduino 提供的库进行基础的开发。

2.1 IO 口操作

2.1.1 IO 口相关操作库函数

在 Arduino 的库中，用于操作 IO 口（Input/Output）的函数主要是 Digital I/O 函数和 Analog I/O 函数。这些函数允许你读取和写入数字信号或模拟信号。

Digital I/O 函数:

pinMode(pin, mode):

用于设置特定引脚的工作模式。pin 是引脚号，mode 可以是以下两个值之一:

INPUT: 将引脚设置为输入模式，用于读取外部数字信号。

OUTPUT: 将引脚设置为输出模式，用于输出数字信号（高电平或低电平）。

digitalRead(pin):

用于读取数字输入引脚的状态，返回值是 HIGH 或 LOW，分别表示引脚处于高电平或低电平状态。

digitalWrite(pin, value):

用于向数字输出引脚写入状态。value 可以是以下两个值之一:

HIGH: 将引脚设置为高电平状态。

LOW: 将引脚设置为低电平状态。

Analog I/O 函数:

analogRead(pin):

用于读取模拟输入引脚的模拟值。Arduino 的模拟输入引脚标记为 A0，A1，...，A5 等。函数返回 0 到 1023 之间的整数，表示引脚上的模拟电压值。

analogWrite(pin, value):

用于输出模拟信号。然而，Arduino 只有一些引脚（如 3, 5, 6, 9, 10 和 11）可以输出模拟信号，而且输出的分辨率为 8 位，即 0 到 255 的整数范围。

其他函数:

void ledcSetup(uint8_t channel, uint32_t frequency, uint8_t resolution);

用于设置 IO 口的 PWM 输出功能，参数解释:

channel：LEDC 的通道号，用于指定要控制的 LED 灯通道。通常，LEDC 具有多个通道，可以控制多个 LED 灯。

frequency：PWM 频率，即周期性变化的脉冲宽度调制信号的频率。它决定了 LED 灯的

闪烁速度。较高的频率会使得 LED 灯看起来更稳定，而较低的频率则会产生闪烁效果。

resolution：分辨率，即 PWM 信号的精度。它决定了 LED 灯的亮度变化范围。较高的分辨率可以提供更平滑的亮度变化，而较低的分辨率则会产生较为明显的亮度变化。

void ledcAttachPin(uint8_t pin, uint8_t channel)

该函数用以将 PWM 输出和 ESP32 指定的 IO 口联系起来

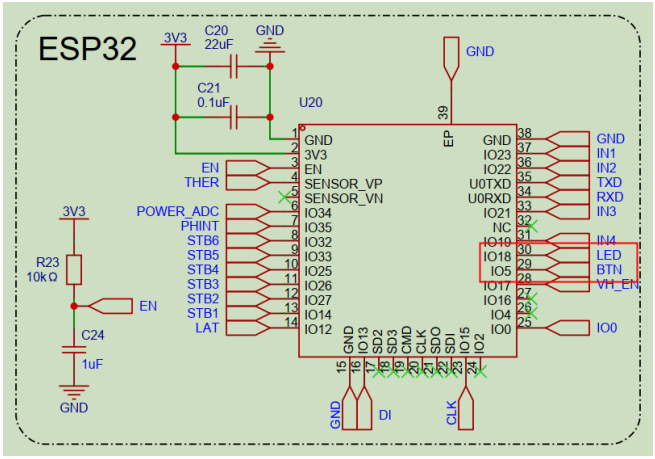
参数解释：

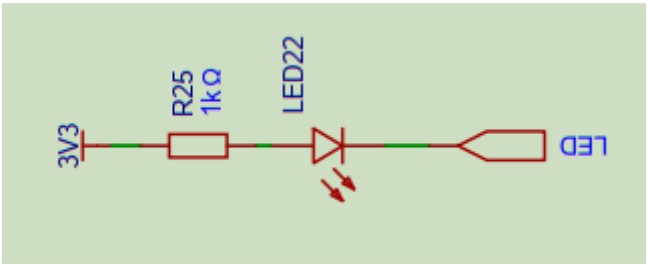
pin：要与 LEDC 控制器连接的引脚号。这将指定用于控制 LED 灯的 GPIO 引脚。

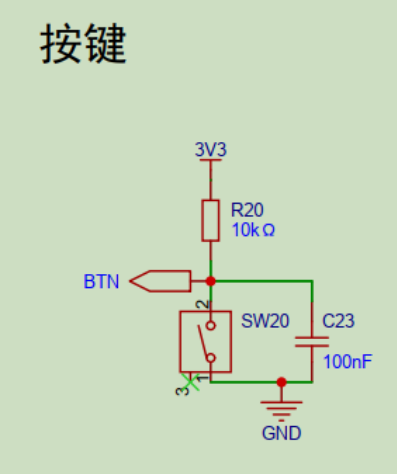
channel：LEDC 通道号，与之前通过 ledcSetup 函数设置的通道号对应。

通过调用 ledcAttachPin 函数，可以将指定的 GPIO 引脚与 LEDC 控制器的一个通道连接起来。这样，通过对该引脚进行写操作，可以控制与该通道对应的 LED 灯。

2.1.2 硬件原理图







2.1.3 按键控制 LED 实现

我们将使用 ESP32 的 GPIO 引脚来控制 LED 灯，同时通过按下按钮来触发 LED 灯的开关。当按键按下一次之后，灯亮 0.5 秒。下方是一个简单的测试代码：

```
/*基础按键测试工程*/
//按键和 LED 的引脚编号
#define BUTTON 5
#define LED 18
#define LED_ON() digitalWrite(LED, LOW)
#define LED_OFF() digitalWrite(LED, HIGH)
//初始化 LED 和按键
void Led_Config(void)
{
   //配置 IO 口模式
   pinMode(BUTTON, INPUT);
   pinMode(LED,OUTPUT);
   //关灯
   LED_OFF();
}
void setup() {
   Led_Config();
   Serial.begin(115200);
   printf("INTI OK\r\n");
}

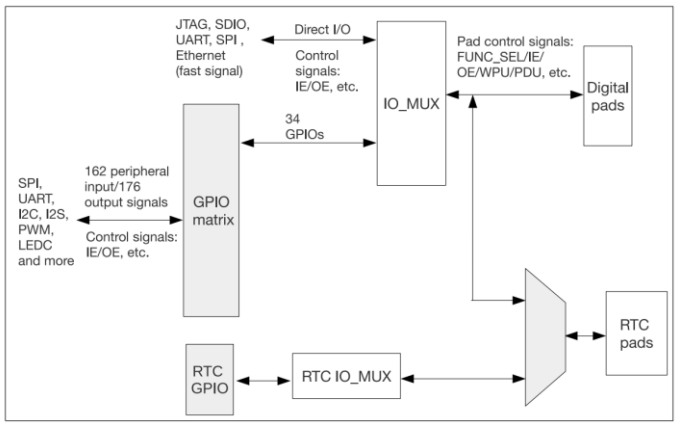
void loop() {
   if(digitalRead(BUTTON)==0)
   {
      printf("LED ON\r\n");
      LED_ON();
```

```
      delay(500);
      printf("LED OFF\r\n");
      LED_OFF();
   }
}
```

2.1.3 ESP32 中的 GPIO 功能补充说明

ESP32 芯片有 34 个物理 GPIO pad。每个 pad 都可用作一个通用 IO，或连接一个内部的外设信号。IO_MUX、RTCIO_MUX 和 GPIO 交换矩阵用于将信号从外设传输至 GPIO pad。这些模块共同组成了芯片的 IO 控制。

下图描述了数字 pad（控制信号： FUNC_SEL、 IE、 OE、 WPU、 WDU 等）和 162 个外设输入以及 176 个外设输出信号（控制信号： SIG_IN_SEL、 SIG_OUT_SEL、 IE、 OE 等）和快速外设输入 / 输出信号（控制信号：IE、 OE 等）以及 RTC IO_MUX 之间的信号选择和连接关系



在 ESP32 芯片中，芯片内部有 34 个 GPIO PAD，Padx 指芯片内部的 IO 引脚，IO_MUX 为复用选择器，用来决定 IO 口的复用功能，一般在 Arduino 环境中编程时会通过 xxxAttachPin 的形式来配置某个 IO 口的复用功能，所以在 ESP32 的开发中，引脚功能的可拓展性要比 STM32 高。

2.2 串口操作

2.2.1 串口相关函数

在 Arduino 中，你可以使用串口（Serial）来进行与计算机或其他外部设备之间的通信。通过串口，你可以发送和接收数据，用于调试、与计算机通信或与其他串口设备交互。以下是 Arduino 中常用的串口相关操作函数：

Serial.begin(baudRate):

该函数用于初始化串口通信。baudRate 参数表示通信的波特率，即每秒传输的位数。常见的波特率包括 9600、115200 等。

Serial.available():

返回接收缓冲区中的字节数。可以通过调用这个函数来检查是否有可用的数据可以读取。

Serial.read():

从串口接收缓冲区中读取一个字节数据并返回。返回值是一个整数（0-255），代表接收到的字节值。如果没有数据可用，则返回-1。

Serial.write(data):

将一个字节数据发送到串口。data 参数是要发送的字节数据。

Serial.print(data), Serial.println(data):

用于向串口发送文本数据。print() 函数发送数据而不换行，println() 函数发送数据并换行。支持多种数据类型的输出，如整数、浮点数、字符、字符串等。你可以使用这些函数来方便地将数据发送到串口进行调试或监视。

Serial.availableForWrite():

返回可以写入的字节数，用于检查串口发送缓冲区是否有足够的空间发送数据。

Serial.flush():

等待将缓冲区中的所有数据发送完成，并清空接收缓冲区。可以在需要确保数据完全发送或清空接收缓冲区时使用。

2.2.2 串口测试案例

下面是一个简单的例子，演示如何在 Arduino 上使用串口进行数据的读取和发送：

```
void setup() {
  Serial.begin(9600); // 初始化串口通信，波特率为 9600
}

void loop() {
  if (Serial.available() > 0) { // 检查是否有数据可读
    int incomingByte = Serial.read(); // 读取数据
    Serial.print("Received: "); // 发送文本
    Serial.println(incomingByte); // 发送读取到的数据
  }
  delay(100); // 等待一段时间，避免频繁读取
}
```

2.3 蓝牙操作

2.3.1 蓝牙相关函数

在 ESP32 使用 Arduino 框架时，可以使用内置的蓝牙库来进行蓝牙操作。使用时需要包含 BluetoothSerial.h 头文件，该库提供了一系列函数，用于在 ESP32 上实现蓝牙 BLE（低功耗蓝牙）和经典蓝牙操作。以下是一些常用的蓝牙操作相关函数:

SerialBT.begin(name):

初始化经典蓝牙功能。name 参数是将要显示在蓝牙设备可见性列表中的设备名称。

SerialBT.available():

检查是否有来自经典蓝牙设备的数据可用。在设备发送数据时，使用此函数来检查是否有数据可读取。

SerialBT.read():

从经典蓝牙设备接收缓冲区中读取一个字节数据并返回。返回值是一个整数 (0-255)，代表接收到的字节值。如果没有数据可用，则返回-1。

SerialBT.write(data):

将一个字节数据发送到经典蓝牙设备。data 参数是要发送的字节数据。

SerialBT.print(data), SerialBT.println(data):

用于向经典蓝牙设备发送文本数据。print() 函数发送数据而不换行，println() 函数发送数据并换行。

SerialBT.end():

停止经典蓝牙功能。在不需要经典蓝牙功能时，可以调用此函数以释放相关资源。

请注意，在使用经典蓝牙功能时，Serial 对象会被替换为 SerialBT 对象。所以，在发送和接收数据时，你应该使用正确的对象。如果你使用 BLE 功能，那么可以继续使用 Serial 对象来进行串口通信。

2.3.2 蓝牙测试代码

在 ESP32 中，经典蓝牙通信可以通过 Serial Bluetooth（SerialBT）库来实现。以下是一个简单的 ESP32 在 Arduino 框架下进行经典蓝牙通信的测试代码。这个例子将创建一个蓝

牙串口设备，可以通过蓝牙串口与其他设备（如手机（安卓手机）或计算机）进行通信。

```
#include <BluetoothSerial.h>

BluetoothSerial SerialBT; // 创建 BluetoothSerial 对象

void setup() {
  Serial.begin(115200);
  SerialBT.begin("MyESP32"); // 初始化蓝牙设备，设备名称为"MyESP32"
}

void loop() {
  // 从蓝牙串口接收数据
  if (SerialBT.available()) {
    char data = SerialBT.read();
    Serial.print("Received data: ");
    Serial.println(data);

    // 向蓝牙串口发送数据
    SerialBT.print("Hello from ESP32!");
  }

  // 一些其他的处理代码
}
```

在这个例子中，我们首先包含了 BluetoothSerial 库并创建了一个 BluetoothSerial 对象 SerialBT。然后，在 setup() 函数中，我们初始化蓝牙设备并指定设备名称为 "MyESP32"，这样其他设备可以通过这个名称来识别并连接蓝牙设备。

在 loop() 函数中，我们使用 SerialBT.available() 来检查是否有来自蓝牙串口的数据可用。如果有数据可用，我们使用 SerialBT.read() 来读取一个字节的数据，并通过串口打印出来。

接着，我们使用 SerialBT.print() 将文本 "Hello from ESP32!" 发送回蓝牙串口，这样其他设备就可以收到这条消息。

请注意，使用经典蓝牙通信时，你可以像使用普通的串口通信一样使用 SerialBT 对象来发送和接收数据。在蓝牙串口设备和其他设备之间进行通信时，确保两者的蓝牙串口连接状态正常。

可以安装资料中提供的蓝牙调试助手先进行测试，通过该软件可以看到蓝牙发送的消息：


卷 (F:) › 蓝牙打印机资料汇总 › 更新20230824 › 蓝牙打印机项目资料 › 3，相关软件 ›
名称
修改日期
类型
大小
PaltFromIO插件
2023/8/30 10:47
文件夹
VSCode插件
2023/8/29 9:40
文件夹
上位机app
2023/8/29 9:33
文件夹
base(1).apk
2023/8/29 16:59
APK 文件
2,132 KB
python-3.11.5-amd64
2023/8/29 10:03
应用程序
25,325 KB

3. 蓝牙打印机功能开发

本章我们将实现蓝牙打印机的功能开发

3.1 打印头控制

3.1.1 打印头介绍

蓝牙打印的打印头兼容 JX-2R-01 和 JX-700-48 两种型号，具有小体积，工作电压宽（3.5~8.5V），效率高的特点，

打印头的具体参数如下图：

项　目	说　明
打印方法	热敏
有效打印宽度（毫米）	48
点密度（点/毫米）	8
打印点数	384 点/行
纸张宽度（毫米）	58
点间距（毫米）	0.125
点大小	0.125mmx0.12mm
最大打印速度	80mm/s(电机驱动电压为 DC 8.5V)
走纸精度	0.0625mm（一个步进距离）
打印头温度侦测	热敏电阻
缺纸侦测	反射光感应器
打印头工作电压(DCV)	3.13~8.5
逻辑工作电压(DCV)	2.7~5.25
电机工作电压(DCV)	3.5~8.5
工作温度	+0℃~50℃（不许有凝露）
工作湿度	20%~85%RH（不许有凝露）
储藏温度	-20℃~60℃（不许有凝露）
储藏湿度	5%~95%RH（不许有凝露）
机械噪音	小于 60dB(A 计权有效值）
胶辊开合次数	大于 5000 次（胶辊离开加复位为一次）
对热敏纸牵引力	≥50g
对热敏纸抓握制动力	≥80g
工作寿命	机构与打印头的耐磨 >50km,打印头的电器寿命为 10^8 个脉冲（在额定状态）
重量(克)	40.5
外形尺寸(长 x 宽 x 高)	72±1.5mm*33±0.5mm*15.5±0.5mm

3.1.2 打印头的工作原理：

将一行 384 个点对应的数据按顺序输入，控制加热信号 STB1、STB2、STB3、STB4、

STB5、STB6，加热打印头，写入的数据中，对应二进制 bit 为 1 的点就会加热成黑点，对应二进制数据为 0 的 bit 则不会变色；与此同时，输入步进电机激励相序信号，转动一步（加热和步进电机转动同时进行）；紧接着输入第二行点的数据……，依次循环 24 次（24*24 字体），完成一整行字符、汉字打印。其内部电路示意图如下所示：

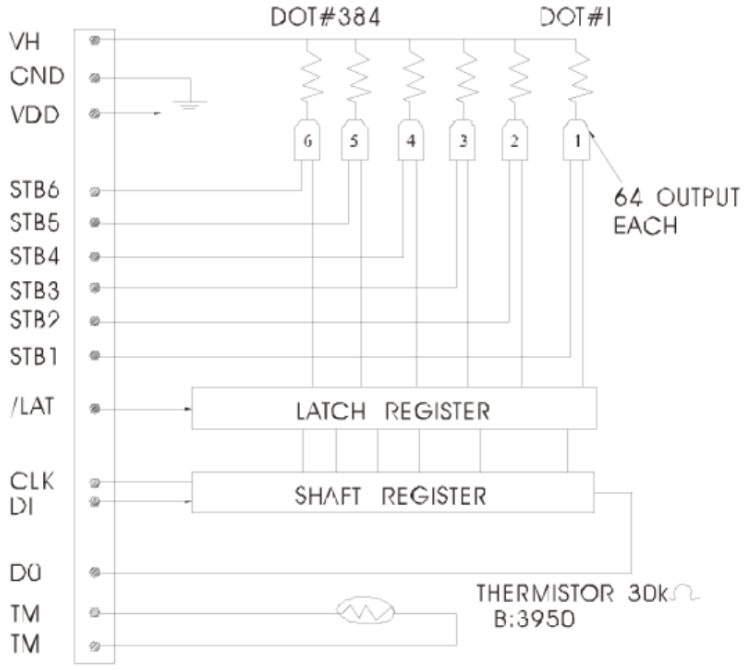


表 8

选通(STB)编号	点编号	点/脚
1	1 到 64	64
2	65 到 128	64
3	129 到 192	64
4	193 到 256	64
5	257 到 320	64
6	321 到 384	64

注意：

1) 每一行数据需要输入 384bits/8=48bytes，如果打印数据不满一行（即少于 48 个字节），则需要填补 0。

2) 停止打印的时候，一定要将步进电机接口关闭，使其线圈没有电流，否则电机会一直发烫。

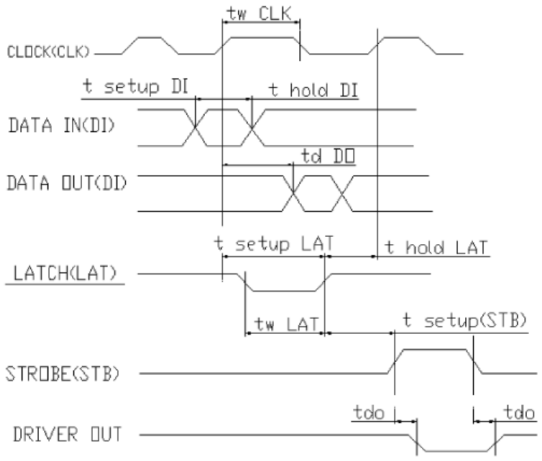
3) 打印头加热时间要把握好，不能太短也不能太长，一般 800us 就 ok；停止打印

或者缺纸的时候， 一

定要将打印头加热控制线全部拉低， 否则， 打印头一直加热， 会降低打印头寿命，甚至烧坏。

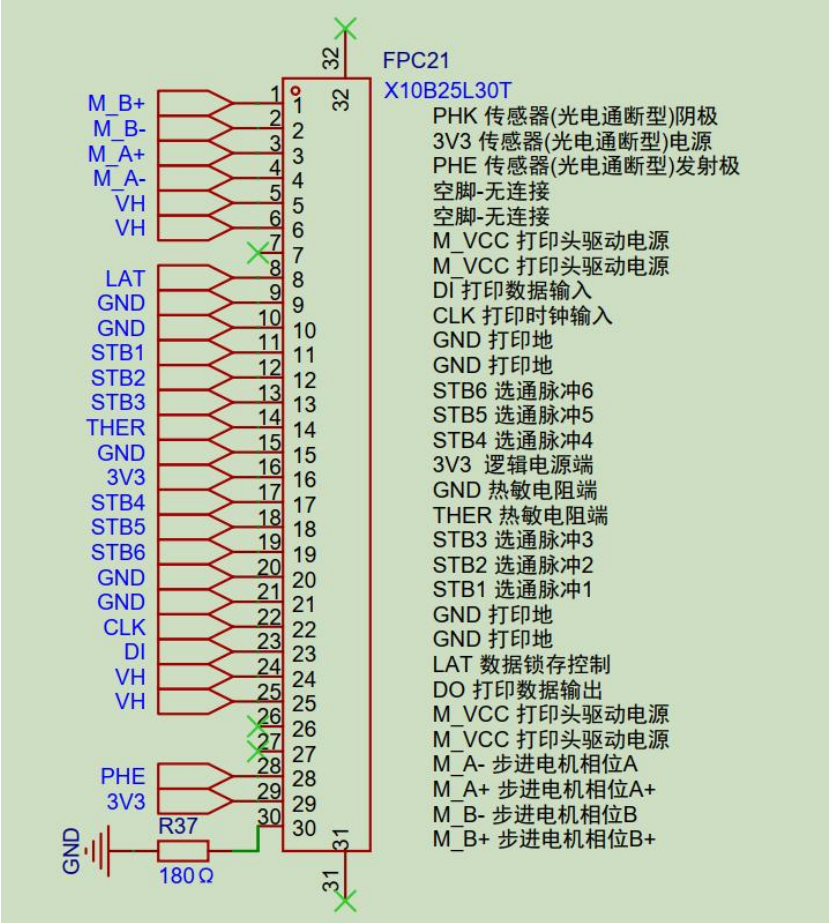
3.1.3 打印头的驱动时序

打印头的通信时序如下，该通信，并采用同步传输的方式，在时钟线 CLK 的时钟周期下，将 DI 数据线上的数据传给打印头，数据发送完成后，通过 LAT 进行数据所存操作，然后控制 STB 进行加热打印即可。在开发时采用 SPI 通信进行数据传输即可。



参 数	代 号	速 度			单位	条 件
		最小	典型	最大		
时钟频率	fCLK	--	--	8	MHz	3≦Vdd≦5.525
		--	--	5	MHz	2.7≦Vdd<3
时钟宽度	twCLK	30	--	--	ns	查询时间表
数据建立时间	testup DI	30	--	--	ns	
数据保持时间	thold DI	30	--	--	ns	
数据延时时间	td DO	--	--	90	ns	3≦Vdd≦5.525
		--	--	130	ns	2.7≦Vdd<3
锁存脉冲宽度	tw LAT	100	--	--	ns	
锁存建立时间	testup LAT	100	--	--	ns	
锁存保持时间	thold LAT	50	--	--	ns	
加热建立时间	testup STB	300	--	--	ns	
输出延时	tdo	--	--	30	μs	5V(Vdd)
		--	--	36	μs	2.7V(Vdd)

3.1.3 打印头硬件接口原理图



3.1.5 打印头驱动

打印头的驱动代码如下：

```
//SPI 参数设置
SPISettings printerSPISettings = SPISettings(1000000, SPI_MSBFIRST, SPI_MODE0);
//SPI 发送数据给打印头
void sendData(uint8_t dataPointer[48]) {

  printerSPI.beginTransaction(printerSPISettings);
  for (uint8_t i = 0; i < 48; ++i) {
    printerSPI.transfer(dataPointer[i]);
  }
  printerSPI.endTransaction();

  digitalWrite(PIN_LAT, 0);
  delayMicroseconds(1);
```

```
    digitalWrite(PIN_LAT, 1);
}
```

内部涉及到的 SPI 相关的库函数，使用时需要先包含 SPI.h 头文件，相关函数功能介绍：

SPI.begin(): 初始化 SPI 总线。在 setup() 函数中调用此函数以启用 SPI 通信。

SPI.beginTransaction(settings): 开始 SPI 通信，设置 SPI 通信参数。settings 参数是 SPISettings 类型的对象，用于指定通信速率、数据位顺序和时钟边沿极性等参数。

SPI.transfer(data): 在 SPI 总线上发送一个字节的数据，并同时接收一个字节的数据。返回值是接收到的字节。

SPI.transfer16(data): 在 SPI 总线上发送一个 16 位的数据，并同时接收一个 16 位的数据。返回值是接收到的 16 位数据。

SPI.transferBytes(data, length): 在 SPI 总线上发送一个字节数组，并接收相同长度的数据数组。data 是一个指向要发送的字节数组的指针，length 是要发送的字节数。

SPI.endTransaction(): 结束 SPI 事务。在 SPI 通信完成后调用此函数。

SPI.end(): 关闭 SPI 总线。在不需要 SPI 通信时，调用此函数以释放相关资源。

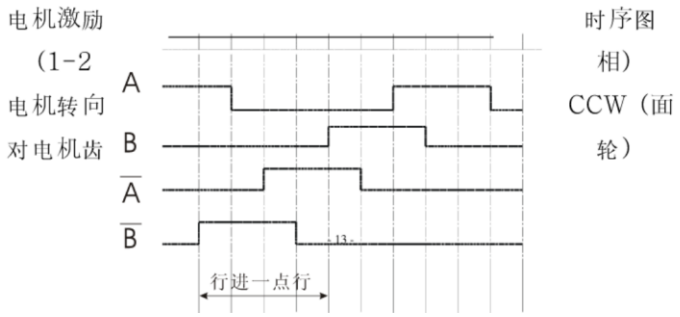
3.1.6 步进电机介绍

打印头所使用的步进电机为两相四线步进电机，步距角为 9°，结合减速齿轮，步进电机每前进一步，可以控制打印纸前进 0.0625mm。其具体参数如下表：

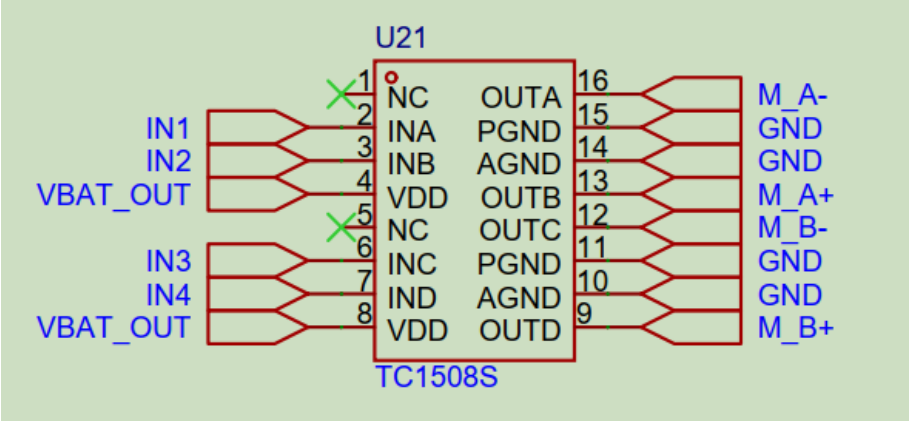
项 目	规 格	条 件
额定电压	3.5~8.5 DCV	
相 位	2 相	
步距角	9º（1-2 相激励）	
步进距离	0.0625 毫米	
相电阻	10 Ω±7%	20℃
相电流	0.357 A	
驱动方式	双极，双相驱动（或 1~2 相）	

打印头的步进电机有四个引脚， 分别连接至电机内部的两组线圈； 可以采用 8 拍驱动方式， 也可以采用 4 拍驱动方式， 驱动时序如下图：

位置	STEP1	STEP2	STEP3	STEP4
A	+	-	-	-
B	-	-	-	-
$\overline{A}$	-	-	+	+
$\overline{B}$	+	+	+	-



3.1.8 步进电机驱动接口原理图



3.1.8 步进电机驱动代码

```
//步进电机节拍
uint8_t motorTable[8][4] = { //JX 系列
    {1, 0, 0, 0},
    {1, 0, 1, 0},
    {0, 0, 1, 0},
    {0, 1, 1, 0},
    {0, 1, 0, 0},
    {0, 1, 0, 1},
    {0, 0, 0, 1},
    {1, 0, 0, 1}
};
```

```
//走纸
void goFront1(uint16_t wait) {
    digitalWrite(PIN_MOTOR_AP, motorTable[motorPos][0]);
    digitalWrite(PIN_MOTOR_AM, motorTable[motorPos][1]);
    digitalWrite(PIN_MOTOR_BP, motorTable[motorPos][2]);
    digitalWrite(PIN_MOTOR_BM, motorTable[motorPos][3]);
    ++motorPos;
    if (motorPos == 8) {
        motorPos = 0;
    }
    delayMicroseconds(wait);
}
```

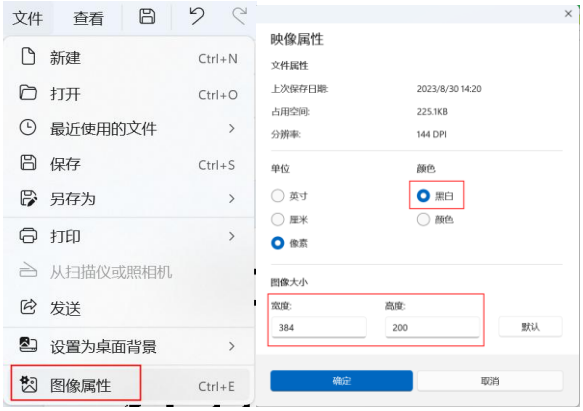
3.1.7 打印图片测试

本项目中使用到的上位机软件会将要打印的所有内容都转化为取模后的数据，这里我们可以先使用取模软件模拟上位机发送的数据。

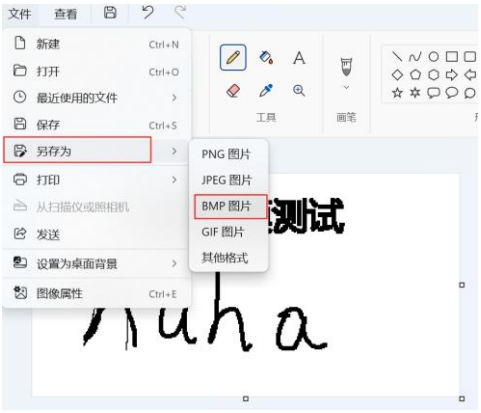
这里我们需要用到的取模软件为：PCtoLCD2002



蓝牙打印机只能支持黑白两种颜色，所以我们需要先准备一个单色的图片，可以借助电脑上的画图软件实现。我们打开画图软件，找到文件中的图像属性一栏，然后在属性中设置图片为黑白，然后修改图片的宽度为固定的 384（打印头一行 384 点）,高度可以根据需求设定，图片越高，取模后的数据越大。



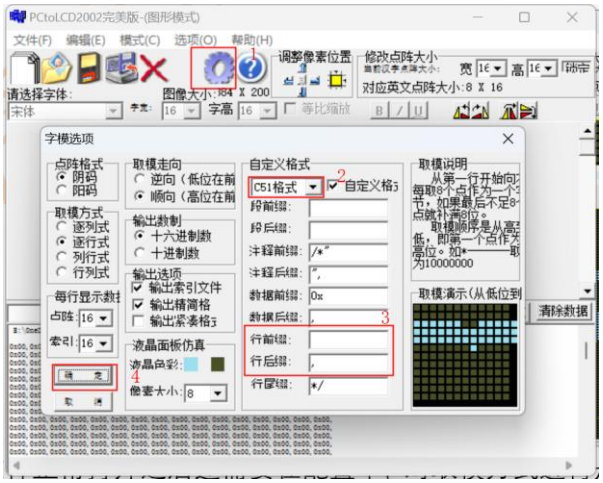
设定好上述信息之后，可以在图中绘制任意自己喜欢的内容，绘制完之后需要将图片保存为 BMP 格式；



保存好之后，通过取模软件打开，需要先选择取模软件的模式为图形模式，然后点击文件，选择打开，找到自己刚保存的位图打开即可在软件中看到输入的图片信息；

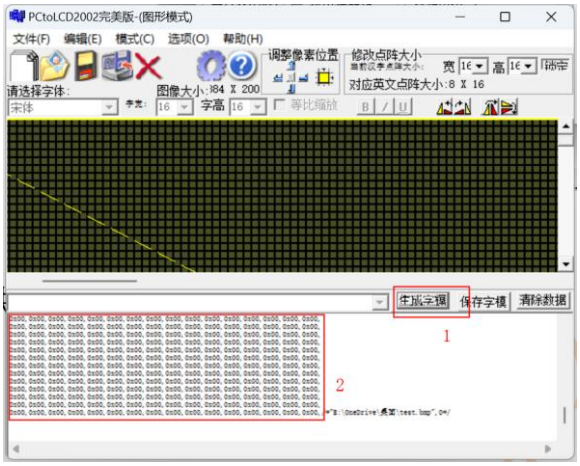


文件正常打开之后还需要在配置中，对取模方式进行是的那个修改，点击齿轮图标进入设置模式，然后按照下图红框所示内容进行修改，修改完成之后点击保存。



保存之后回到主界面，然后点击生成字模按钮，会将图片的模型信息显示在下图所示的红框 2 位置。可以在工程中单独创建一个文件，用以保存图片模型数据，将生成的内容复制

到自己的文件里即可。




```h
#ifndef __IMG_H
#define __IMG_H

#include "Arduino.h"

extern const uint8_t img_data[];

#endif
```

```cpp
#include <img.h>

const uint8_t img_data[]={0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
```

打印头打印测试核心代码：

```
//打印测试函数：
void Print_test(void)
{
    uint32_t cleardata=0;
    for (cleardata = 0; cleardata < 9600; ++cleardata)
    {
        printData[cleardata] = img_data[cleardata];
        printDataCount++;
    }
    startPrint();
    goFront(20, MOTOR_TIME);
}
```

打印头驱动代码：

```
void startPrint()
{
    int pinnumber;
    int i;
    int cache;//分批打印标志，避免同时开启所有打印头
    onprint = 1;
```

```
  PowerONTime = millis();
  if (PaperSta == 1) {
    static unsigned char motor_add = 0;
    Serial.println("[INFO]正在打印...");
    Serial.printf("[INFO]共%u 行\n", printDataCount / 48);
    digitalWrite(PIN_VHEN, 1);
    //digitalWrite(PIN_STATUS, 1);
    cache = -1;
    for (uint32_t pointer = 0; pointer < printDataCount; pointer += 48) {
      cache = -1;
      while (cache != 0) {//分批加热打印头
        if (cache == -1) {
          cache = 0;
        }
        pinnumber = 0;
        for (i = cache; i < 48; i++) {
          //判断要打印的一行数据中是否有需要打印的点，如果没有后续就不再加热打印头
          //这里用了 excel 公式做了个数组来统计 0-255 转换成二进制后 1 的个数输出当前行打印点数，也可通过算法实现
          pinnumber = pinnumber + shuzu16to2jingzhi[printData[pointer + i]];
          if (pinnumber <= printpin) {//判断打印的数据中要加热的点数是否超过规定最大值，如果超过了就先加热指定的点
            printDatacache[i] = printData[pointer + i];
            if (i == 47) {
              cache = 0;
            }
          } else {
            cache = i;
            break;
          }
        }
        sendData(printDatacache);
        for (int j = 0; j < 48; j++) {
          printDatacache[j] = 0;
        }
        if (pinnumber != 0) {
          digitalWrite(PIN_STB1, 1);
          digitalWrite(PIN_STB2, 1);
          digitalWrite(PIN_STB3, 1);
          digitalWrite(PIN_STB4, 1);
          digitalWrite(PIN_STB5, 1);
          digitalWrite(PIN_STB6, 1);
          delayMicroseconds((PRINT_TIME) * ((double)heat_density / 100));
```

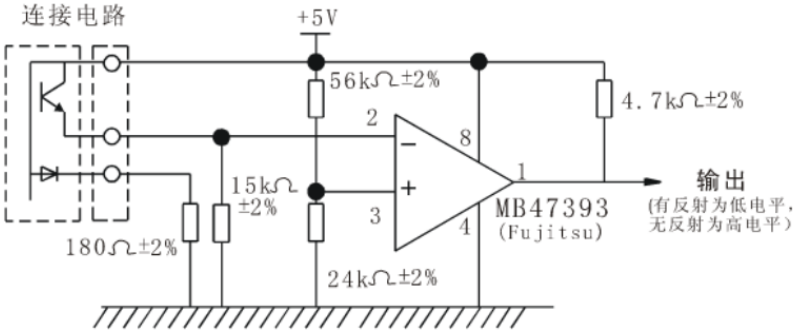
```
                digitalWrite(PIN_STB1, 0);
                digitalWrite(PIN_STB2, 0);
                digitalWrite(PIN_STB3, 0);
                digitalWrite(PIN_STB4, 0);
                digitalWrite(PIN_STB5, 0);
                digitalWrite(PIN_STB6, 0);
                delayMicroseconds(PRINT_TIME_);
              }
            }
            goFront1(MOTOR_TIME);
            goFront1(MOTOR_TIME);
            goFront1(MOTOR_TIME);
            goFront1(0);
        }
        digitalWrite(PIN_MOTOR_AP, 0);
        digitalWrite(PIN_MOTOR_AM, 0);
        digitalWrite(PIN_MOTOR_BP, 0);
        digitalWrite(PIN_MOTOR_BM, 0);
        clearSTB();
        clearData();
        printDataCount = 0;
        Serial.println("[INFO]打印完成");
        digitalWrite(PIN_VHEN, 0);
        PowerONTime = millis();
        onprint = 0;
    } else {
            startBeep();
            delay(500);
            stopBeep();
            delay(500);
            startBeep();
            delay(500);
            stopBeep();
    }
}
```

3.2 缺纸检测

3.2.1 缺纸检测原理

JX-2R-01 机芯采用一个反射性光电通断检测传感器，可以用来实现缺纸检测或纸张定

位，如下图所示，缺纸时传感器光无法被反射输出高电平，正常时传感器光被反射，输出低电平。缺纸时不要进行打印操作，容易烧毁打印头。



3.2.2 缺纸检测代码

通过读取缺纸检测引脚的高低电平，来判断是否有纸，缺纸时进行蜂鸣器报警。

```
//缺纸检测
void papertest() {
   if ((digitalRead(PIN_PTEST) == 1) ) { //缺纸检测
      startBeep();
      delay(500);
      stopBeep();
      delay(500);
      //printf("缺纸");
      digitalWrite(PIN_VHEN, 0);
      PaperSta = 0;
   }
   else {
      digitalWrite(PIN_VHEN, 1);
      stopBeep();
      //printf("有纸");
      PaperSta = 1;
   }
}
```

3.3 电压检测

3.3.1 电压检测

项目采用 18650 锂电池供电，满电电压 4.2V，欠电电压 3.7V。通过 ADC 采集可以实现电压的检测。放置过放损坏电池，影响打印效果。通过 ADC 去读取电压检测引脚的数值，

结合电压检测电路，可以求得大致的电压值。

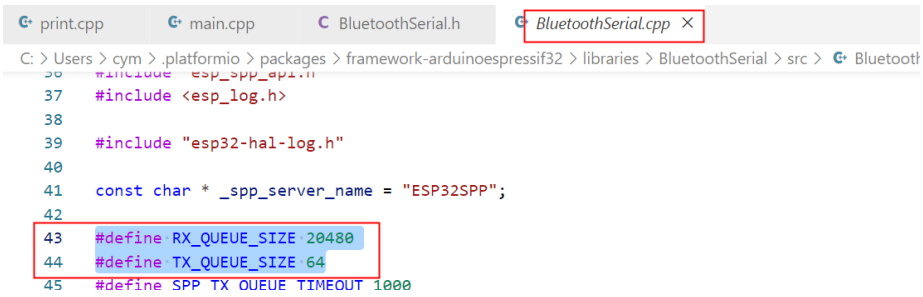
3.3.2 代码实现

```
void BatteryPower() {
  int i;
  digitalWrite(PIN_BATTEST, 0);
  for (i = 0; i < 10; i++) {
    adc = analogRead(PIN_BATV);
    adcc += adc;
  }
  digitalWrite(PIN_BATTEST, 1);
  adc = adcc / 10;
  batVoltage = (adc * 6.6 * Voltagejiaozhun / 4095); //电压
  Serial.print("电池电压:"); Serial.println(batVoltage);
  for (i = 100; i > -1; i--) {
    if (batVoltagelist[i] <= batVoltage) {
      PRINTER_BATTERY = i;
      break;
    }
  }
  adc = 0;
  adcc = 0;
  Serial.print("电量"); Serial.print(PRINTER_BATTERY); Serial.println("%");
}
```

3.4 上位机通信

蓝牙打印机的上位机采用的是公版喵喵机 App，通过蓝牙和上位机进行连接，所以需要解析喵喵机 app 的蓝牙通信协议。由于软件的不开源性，所以只能通过 app 逆向的方法破解该 app 的通信协议。

蓝牙打印机在使用时会发送大量的数据需要进行缓存，主控采用 ESP32-WROOM 时所能申请到的保存数据的空间为 60KB，打印 1280 行左右，当数据超出时会出现打印数据丢失的问题。并且需要修改蓝牙库中接收和发送数据的大小，具体修改内容和修改位置如下:



3.4.1APP 通信协议分析

通过 App 逆向，可以获取到，蓝牙通信的协议格式如下：

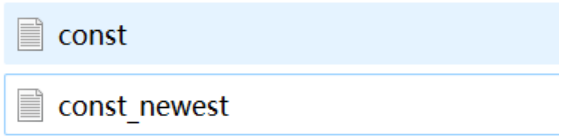
```
(1 byte) 2
(1 byte) 控制命令
(1 byte) 当前 payload 包序列号
(2 byte) payload 长度
(n byte) payload
(4 byte) payload 的 CRC32 校验码
(1 byte) 3
```

当 payload 过长时，拆成长度不大于 2016 的多个包，依次发送。

其中 CRC32 校验码的初始值为 crcKey = 0x35769521。控制命令包括 PRT_PRINT_DATA 等数字常量，1.0.2 版本 Apk 中有 26 个，2.0.3 版本中增加到 47 个，向下兼容。

机器返回的消息同样遵循上述协议，当最后一个包的数据字段为 0x00 的时候表示信息发送完毕

控制命令参考资料里的文档：



常用的指令和功能如下：

#define PRINT_DATA	0	//打印数据
#define PRINT_DATA_COMPRESS	1	//打印数据压缩
#define FIRMWARE_DATA	2	//固件数据
#define USB_UPDATE_FIRMWARE	3	//USB 更新固件
#define GET_VERSION	4	//获取版本
#define SENT_VERSION	5	//发送版本
#define GET_MODEL	6	//获取模式
#define SENT_MODEL	7	//发送模式
#define GET_BT_MAC	8	//获取蓝牙 MAC

```
#define SENT_BT_MAC                 9       //发送蓝牙 MAC
#define GET_SN                      10      //获取 SN 码
#define SENT_SN                     11      //发送 SN 码
#define GET_STATUS                  12      //获取状态
#define SENT_STATUS                 13      //发送状态
#define GET_VOLTAGE                 14      //获取电压
#define SENT_VOLTAGE                15      //发送电压
#define GET_BAT_STATUS              16      //获取电量状态
#define SENT_BAT_STATUS             17      //发送蓝牙状态
#define GET_TEMP                    18      //获取打印头温度
#define SENT_TEMP                   19      //发送打印头温度
#define SET_FACTORY_STATUS          20      //设置出厂状态
#define GET_FACTORY_STATUS          21      //获取出厂状态
#define SENT_FACTORY_STATUS         22      //发送出厂状态
#define SENT_BT_STATUS              23      //发送蓝牙状态
#define SET_CRC_KEY                 24      //设置 CRC 密钥
#define SET_HEAT_DENSITY            25      //设定热密度
#define FEED_LINE                   26      //进料线
#define PRINT_TEST_PAGE             27      //打印测试页
#define GET_HEAT_DENSITY            28      //获取热密度
#define SENT_HEAT_DENSITY           29      //发送热密度
#define SET_POWER_DOWN_TIME         30      //设定关机时间
#define GET_POWER_DOWN_TIME         31      //获取断电时间
#define SENT_POWER_DOWN_TIME        32      //发送断电时间
#define FEED_TO_HEAD_LINE           33      //送至标题行
#define PRINT_DEFAULT_PARA          34      //打印默认参数
#define GET_BOARD_VERSION           35      //获取板子版本
#define SENT_BOARD_VERSION          36      //发送板子版本
#define GET_HW_INFO                 37      //获取硬件信息
#define SENT_HW_INFO                38      //发送硬件信息
#define SET_MAX_GAP_LENGTH          39      //设定最大间隙长度
#define GET_MAX_GAP_LENGTH          40      //获取最大间隙长度
#define SENT_MAX_GAP_LENGTH         41      //发送最大间隙长度
#define GET_PAPER_TYPE              42      //获取纸张类型
#define SENT_PAPER_TYPE             43      //发送纸张类型
#define SET_PAPER_TYPE              44      //设置纸张类型
#define GET_COUNTRY_NAME            45      //获取国家名称
#define SENT_COUNTRY_NAME           46      //发送国家名称
#define DISCONNECT_BT               47      //断开蓝牙
#define GET_DEV_NAME                48      //获取设备名称
#define SENT_DEV_NAME               49      //发送设备名称
```

上位机和设备建立链接时需要发送的指令和顺序如下：

1， 上位机发送 24 指令，设置 CRC 校验的密钥，固定四字节 crcKey = 0x35769521

2， 发送 10 指令获取设备的 SN 码，需要蓝牙返回指定的 SN 码："P1001705253855"
3， 上位机发送 66 指令，设备返回固定字符串信息："BK3432"，蓝牙设备型号
4， 上位机发送 127 指令，设备返回固定信息：{0x76, 0x33, 0x2e, 0x33, 0x38, 0x2e, 0x31, 0x39, 0, 0, 0, 0};
5， 上位机发送 25 指令，设定打印机加热时间（热密度）
6， 上位机发送 48 指令，请求设备名称，设备返回自己设定的名字
7， 上位机发送 16 指令，请求设备电池信息，设备返回当前电压
8， 上位机发送 31 指令，请求关机时间，设备返回设定的关机时间默认 3600 分钟
9， 上位机发送 4 指令，获取打印机版本信息，默认版本 2.0.1，设备返回{0x01, 0x00, 0x02};
10， 上位机发送 44 指令，设定纸张类型，设备纸张固定，不需要修改

上述流程中，需要注意设备返回的数据，在开发蓝牙打印机的功能时，需要正确返回数据，否则会导致设备连接失败，并且每条指令在接受完成之后，设备都需要进行回复，回复内容固定为 00，类型根据接收到的类型来定。如果指令没有正确回应，软件会重复发送直至超时。

上位机和设备之间传输打印数据发送的指令和数据流程如下：
1， 打印时会先发送两次 25 指令用以设置打印热密度，也就是打印加热时间。
2， 发送 26（进料线）指令，也就是开始打印指令
3， 然后发送 0 指令，开始传输数据，分包发送打印的数据，一包最大 1920
4， 数据发送完毕之后，会再次发送 26 指令，收到之后，打印机开始打印数据。

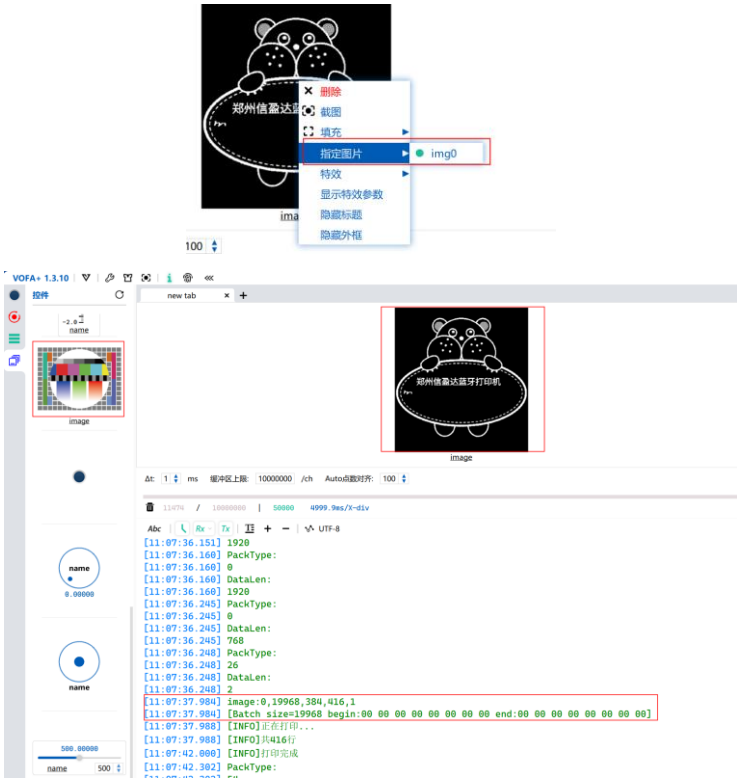
蓝牙打印机打印数据分析：

喵喵机上位机软件会提前将要打印的数据进行取模，打印机收到的数据已经是处理好的数据，我们只需要根据数据进行打印即可。这里我们可以通过 vofa 调试助手来看一下打印的数据：

在开始打印之前我们可以先通过串口把图片的数据输出到 vofa 串口助手里，具体代码如下：

```
179     //进料线
180     case FEED_LINE:
181       if (printDataCount / 48 != 0) {
182         printf("image:%d,%d,%d,%d,%d\n",IMG_ID,printDataCount,IMG_WIDTH,printDataCount/48,IMG_FORMAT);
183         for(i=0;i<printDataCount;i++)
184         {
185           Serial.write(printData[i]);
186         }
187         startPrint();
188         ongofront = 1;
189         goFront(Finish_Out, MOTOR_TIME);     //打印完毕出纸长度，单位：点，8点/mm。
190         ongofront = 0;
191       }
192       printDataCount = 0;
193       break;
```

数据输出之后我们还需要再软件中添加一个图片组件，然后选择数据源为 IMG0 通道，就可以再软件中显示对应的数据。



3.4.3 上位机通信代码实现

蓝牙发送代码：

```
void paperang_send(void)
{
    SerialBT.write(dataPack, dataPack_len);
}
//蓝牙回复指令
void paperang_send_ack(uint8_t type)
{
    uint8_t ackcrc = 0;
    dataPack[0] = START_BYTE;
    dataPack[1] = type;
    dataPack[2] = 0x00;
    dataPack[3] = 0x01;
    dataPack[4] = 0x00;
    dataPack[5] = 0x00;
    crc32_result = crc32.calc(&ackcrc, 1);
    memcpy(dataPack + 6, (uint8_t *)&crc32_result, 4);
    dataPack[10] = END_BYTE;
    dataPack_len = 11;
    paperang_send();
}
```

```
//蓝牙回复信息
void paperang_send_msg(uint8_t type, const uint8_t* dat, uint16_t len)
{
  dataPack[0] = START_BYTE;
  dataPack[1] = type;
  dataPack[2] = 0x00;
  memcpy(dataPack + 3, (uint8_t *)&len, 2);
  memcpy(dataPack + 5, dat, len);
  dataPack_len = 5 + len;
  crc32_result = crc32.calc(dat, len);
  memcpy(dataPack + dataPack_len, (uint8_t *)&crc32_result, 4);
  dataPack[dataPack_len + 4] = END_BYTE;
  dataPack_len += 5;
  paperang_send();
}
```

整体数据获取流程如下：

```
while (1) {
    if (SerialBT.available()) {//判断蓝牙串口是否收到数据
      PowerONTime = millis();//更新自动关机时间
      c = SerialBT.read();//读取蓝牙数据
      if (c == START_BYTE && gotStartByte == 0)//判断收到的数据是否是开始码、以及是否是第一次收到数据
      {
        gotStartByte = 1;//开始接收数据的标志位
        readpos = 0;//接收状态判断
      }
      else if (readpos == 1)//判断接收到的第二个字节数据、也就是指令的类型
      {
        packHeader.packType = c;
        Serial.println("packType:");
        Serial.println(packHeader.packType);
        packHeader.packIndex = SerialBT.read();//读取数据包序列号
        packHeader.dataLen = SerialBT.read();//读取数据包长度
        packHeader.dataLen += SerialBT.read() << 8;//读取数据包长度
        Serial.println("dataLen:");
        Serial.println(packHeader.dataLen);
      }
      else if (readpos == 2 && packHeader.dataLen != 0 && packHeader.dataLen < 2048)//开始接收数据部分内容
      {
        i = packHeader.dataLen - 1;//由于在循环时 220 行已经接收过一字节数据，所以再次接收数据时长度需要减去 1
        if (packHeader.packType == PRINT_DATA)//判断收到的指令是否是打印数据
```

```
            {
              printData[printDataCount++] = c;//先保存之前收到的第一字节数据
              while (i) {//循环接收剩余数据
                while (SerialBT.available() == 0);
                printData[printDataCount++] = SerialBT.read();
                --i;
              }
            }
            else//不是打印数据指令时，保存收到的指令内容
            {
              dataPack_read[dataPack_read_pos++] = c;
              while (i) {
                dataPack_read[dataPack_read_pos++] = SerialBT.read();
                --i;
              }
            }
          }
          else if (readpos < 7)
          {
            //接收 CRC 校验数据
          }
          else if (c == END_BYTE && readpos == 7)//判断接收到包尾标志，且 readpos 为 7
代表接收完成
          {
            paperang_process_data();//处理数据
            gotStartByte = 0;
            dataPack_read_pos = 0;
            readpos = 0;
            packHeader.dataLen = 0;
          }
          else
          {
            Serial.println("ERROR");
            gotStartByte = 0;
            dataPack_read_pos = 0;
            readpos = 0;
            packHeader.dataLen = 0;
          }
          if (gotStartByte == 1)
            ++readpos;
        } else {
          Powertest();
          papertest();
          delay(1);
```

```
    }
  }
```

不同指令的处理代码如下：这里由于喵喵机指令回复固定，所以不建议开发者自己修改回复指令，否则会导致上位机通信失败：

```
void paperang_process_data()
{
  uint32_t tmp32;
  switch (packHeader.packType)
  {

    case PRINT_DATA:
      return;
    //设置 CRC 密钥
    case SET_CRC_KEY:
      tmp32 = dataPack_read[0] << 24 + dataPack_read[1] << 16 + dataPack_read[2] <<
8 + dataPack_read[3];
      crc32.init(tmp32);
      break;
    //获取版本信息并发送
    case GET_VERSION:
      paperang_send_msg(SENT_VERSION, PRINTER_VERSION, 3);
      break;
    //获取设备名称并发送
    case GET_DEV_NAME:
      paperang_send_msg(SENT_DEV_NAME, (uint8_t *)PRINTER_NAME, 2);
      break;
    //获取 SN 码并发送
    case GET_SN:
      paperang_send_msg(SENT_SN, (uint8_t *)PRINTER_SN, strlen((char *)PRINTER_SN));
    //获取电池电量并发送
    case GET_BAT_STATUS:
      BatteryPower();
      paperang_send_msg(SENT_BAT_STATUS, &PRINTER_BATTERY, 1);
      break;
    //获取国家名称并发送
    case GET_COUNTRY_NAME:
      paperang_send_msg(SENT_COUNTRY_NAME, (uint8_t *)COUNTRY_NAME, 2);
      break;
    case CMD_42:
      paperang_send_msg(CMD_43, (uint8_t *)CMD_42_DATA, strlen(CMD_42_DATA));
      break;
    case CMD_7F:
```

```
        paperang_send_msg(CMD_80, CMD_7F_DATA, 12);
        break;
    case CMD_81:
        paperang_send_msg(CMD_82, CMD_81_DATA, 16);
        break;
    case CMD_40:
        paperang_send_msg(CMD_41, CMD_40_DATA, 1);
        break;
    //设置断电时间
    case SET_POWER_DOWN_TIME:
        power_down_time = dataPack_read[0] << 8 + dataPack_read[1];
        if (power_down_time == 90112) {
            PowerOFFTime = "600000";
            Serial.print("自动关机时间设置为：10 分钟");
        }
        else if (power_down_time == 262144) {
            PowerOFFTime = "1800000";
            Serial.print("自动关机时间设置为：30 分钟");
        }
        else if (power_down_time == 67108864) {
            PowerOFFTime = "3600000";
            Serial.print("自动关机时间设置为：1 小时");
        }
        else if (power_down_time == 12582912) {
            PowerOFFTime = "10800000";
            Serial.print("自动关机时间设置为：3 小时");
        }
        else if (power_down_time == 1310720) {
            PowerOFFTime = "18000000";
            Serial.print("自动关机时间设置为：5 小时");
        }
        else if (power_down_time == 12582912) {
            PowerOFFTime = "36000000";
            Serial.print("自动关机时间设置为：10 小时");
        }
        else if (power_down_time == 1310720) {
            PowerOFFTime = "9999999999";
            Serial.print("关闭自动关机");
        }
        set_string(PwrOFFtime, PwrOFFtime + 1, PowerOFFTime, 1);
        break;
    //发送断电时间并发送
    case GET_POWER_DOWN_TIME:
        paperang_send_msg(SENT_POWER_DOWN_TIME,  (uint8_t  *)&power_down_time,
```

```
2);
            break;
        //设置打印密度
        case SET_HEAT_DENSITY:
            heat_density = dataPack_read[0];
            break;
        //获取打印密度并发送
        case GET_HEAT_DENSITY:
            paperang_send_msg(SENT_HEAT_DENSITY, &heat_density, 1);
            break;
        //获取打印头温度
        case GET_TEMP:
            HeatTemp();
            paperang_send_msg(SENT_TEMP, &head_temp, 1);
            break;
        //进料线
        case FEED_LINE:
            if (printDataCount / 48 != 0) {
                startPrint();
                ongofront = 1;
                goFront(Finish_Out, MOTOR_TIME);      //打印完毕出纸长度，单位：点，8 点
/mm。
                ongofront = 0;
            }
            printDataCount = 0;
            break;
        default:
            break;

    }
    paperang_send_ack(packHeader.packType);
}
```

3.5 喵喵机 APP 使用

首先安装资料中提供的蓝牙打印机的 app，仅支持安卓手机。安装完成之后，直接注册登录即可。注意不要更新软件，更新后会导致 app 通信协议变更，无法正常通信。



软件使用流程：

1，打开软件之后会进入登录界面，这里可以选择直接用手机号登录，或者使用离线登录，登陆后功能更加完整，支持拍照打印等；



2，使用离线登录之后即可进入离线模式，功能相对单一，界面如下；



3，链接设备，设备正常上电之后，点击软件中的添加设备按钮，如果设备正常工作，就会在软件中看到一个名为“MaoPaperang”的设备。如果搜索不到该设备可以点击刷新按钮，或者检查代码中是否正确修改 ESP32 模块的蓝牙设备类型。



4，链接成功之后会返回主界面，可以看到刚才链接的设备，如果能搜索到，但是链接不成功，则需要检查手机是否联网，或者代码中上位机指令接收和回应是否和笔记中的一致。



5，点击右上角的设置功能可以看到，设备的基本信息如下：



6，打印时可以选择第四步中任意一种，这里以打印文本为例，在文本框中输入自己想要打印的内容，然后点击右上角的打印图标，会进入打印预览界面，点击立即打印即可开始打印数据。

